ESPACIO RESERVADO

Insertar aquí el logo oficial de Areandina

PAY SERVICES

Actividad Evaluativa Eje 3

Desarrollo de aplicación

Fundación Universitaria del Área Andina

Gestión de Bases de Datos

Docente: Yashin Alberto Rueda Matos

Integrantes

Santiago | Sebastian | Wilmer Dávila Ríos

9 de noviembre de 2026

1. Introducción

Pay Services es una aplicación web académica para gestionar registros simulados de pagos de servicios. El proyecto integra una interfaz web, una API REST en Node.js y Express y persistencia relacional en MySQL sobre Aiven. No procesa dinero real.

2. Objetivo

Desarrollar un CRUD que permita registrar, listar, consultar, actualizar y eliminar pagos; mostrar cómo filas y columnas relacionales se convierten en objetos JavaScript; y documentar la conexión segura entre la aplicación y MySQL.

3. Tecnologías

Node.js, Express, `mysql2/promise`, `dotenv`, MySQL y Aiven. El cliente está escrito en HTML, CSS y JavaScript nativo y consume la API con `fetch`.

4. Modelo relacional real

La inspección de Aiven confirmó que `defaultdb` no tenía tablas reutilizables. Se creó la tabla mínima `pagos` con `CREATE TABLE IF NOT EXISTS`; no se borró ni modificó ninguna tabla existente. La tabla no tiene claves foráneas.

Columna	Tipo	Regla observada
`id`	`INT`	Clave primaria, autoincremental
`cliente`	`VARCHAR(100)`	Obligatorio
`servicio`	`VARCHAR(60)`	Obligatorio
`referencia`	`VARCHAR(60)`	Obligatorio
`valor`	`DECIMAL(12,2)`	Obligatorio; debe ser positivo
`fecha_pago`	`DATE`	Obligatorio
`estado`	`VARCHAR(20)`	Obligatorio; `Pendiente` o `Pagado`

5. Mapeo relacional a objetos

La aplicación consulta filas MySQL y las entrega como objetos JSON. La tabla `pagos` corresponde a la colección de pagos; una fila a un objeto; cada columna a una propiedad; y la clave primaria al identificador `id`. No hay claves foráneas en este modelo mínimo. Las rutas ejecutan `INSERT`, `SELECT`, `UPDATE` y `DELETE` explícitos con parámetros; no se usa un ORM.

6. Arquitectura y conexión

El código queda dividido en `backend/` y `frontend/`. El servidor Express en `backend/` sirve el frontend desde `/` y la API desde `/api`; al estar en el mismo dominio, el navegador no necesita CORS ni una URL de API aparte. `backend/src/` separa rutas, controladores, modelo y conexión MySQL. El frontend usa HTML, CSS y JavaScript nativo para reducir dependencias. `mysql2/promise` mantiene un pool pequeño. La conexión usa TLS validado y puede recibir la CA de Aiven mediante `DB_CA_PATH`. Las variables privadas se leen de `backend/.env`, excluido de Git. La ruta `/api/health` devuelve estado sin credenciales.

La conexión real a `defaultdb` se comprobó con `npm run db:inspect`. El hostname se corrigió a partir de Workbench y el certificado CA se instaló localmente en `backend/certs/ca.pem`; `DB_CA_PATH=certs/ca.pem` mantiene la validación TLS activa. La contraseña permanece solo en `backend/.env`, excluido de Git.

7. API REST

Método y ruta	Función	Respuesta prevista
`GET /api/pagos`	Listar	`200`
`GET /api/pagos/:id`	Consultar	`200`, `400` o `404`
`POST /api/pagos`	Crear	`201` o `400`
`PUT /api/pagos/:id`	Actualizar	`200`, `400` o `404`
`DELETE /api/pagos/:id`	Eliminar con confirmación del cliente	`204`, `400` o `404`
`GET /api/health`	Comprobar aplicación y base	`200` o `503`

Los campos del formulario incluyen cliente, servicio, referencia, valor, fecha y estado. El backend valida campos, importes, fecha y opciones permitidas antes de consultar la base.

8. Interfaz

La interfaz en español contiene formulario para crear y editar, lista de pagos, edición, confirmación antes de borrar, mensajes, carga y estado vacío. Los importes se presentan en COP. El módulo **Conectar / reintentar** consulta `/api/health`, muestra el estado de Render y Aiven e intenta de nuevo por un periodo limitado si el servicio gratuito está despertando. No mantiene pings activos en segundo plano. La conexión y el CRUD real están verificados; todavía no se han tomado capturas de pantalla para `evidencias/`.

9. Pruebas realizadas

Pruebas unitarias locales: **6 aprobadas, 0 fallidas** con `npm test`. La inspección real confirmó la tabla y las siete columnas. `node scripts/smoke-crud.js` completó crear, consultar, listar, actualizar, volver a leer desde MySQL y eliminar, con respuestas HTTP esperadas; al final confirmó `404` para el registro eliminado. Además, se insertó un registro temporal, se reinició el servidor, se recuperó desde Aiven y luego se eliminó. No quedaron registros temporales de estas pruebas.

10. Ejecución local

Desde la carpeta raíz, entrar a `backend/`, instalar con `npm install`, completar `backend/.env` y colocar el certificado CA en `backend/certs/ca.pem`. Ejecutar `npm run db:inspect` para revisar la base; si no existe una tabla reutilizable, `npm run db:setup` aplica únicamente `CREATE TABLE IF NOT EXISTS`. Luego iniciar con `npm start`; la página queda disponible en `http://localhost:3000`.

11. Preparación del despliegue

El proyecto incluye `start`, escucha `process.env.PORT`, sirve frontend y API desde Express y no tiene dependencias de despliegue separadas. En Render se puede usar `cd backend && npm ci` para instalar y `cd backend && npm start` para arrancar. Las variables privadas se configuran en Environment; la CA se agrega como Secret File `ca.pem` y `DB_CA_PATH` se define como `/etc/secrets/ca.pem`. Render expone los Secret Files bajo `/etc/secrets/` durante la ejecución (documentación oficial). En el plan gratuito Render duerme los servicios tras 15 minutos sin tráfico y los reactiva con la siguiente solicitud; el botón de conexión permite reintentar y comprobar la base durante ese arranque (límites del plan gratuito). El proyecto sí se desplegó en `https://aiveneje3.onrender.com`; la ruta `/api/health` se comprobó conectada.

12. Variables de entorno

La configuración privada queda en `backend/.env`; no se guardan variables en el frontend ni en el repositorio. La contraseña de Aiven está solo en `DB_PASSWORD` del `.env` local. `DB_HOST`, `DB_PORT`, `DB_USER` y `DB_NAME=defaultdb` se verificaron con los datos de conexión y la inspección. `DB_SSL=true` mantiene TLS y `DB_CA_PATH=certs/ca.pem` indica al driver dónde cargar la CA en el entorno local. En Render, el archivo se agrega como Secret File y la ruta se cambia a `/etc/secrets/ca.pem`.

Variable	Uso
`PORT`	Puerto del servidor web; local por defecto `3000`
`DB_HOST`	Servidor MySQL de Aiven
`DB_PORT`	Puerto MySQL de Aiven
`DB_USER`	Usuario de MySQL
`DB_PASSWORD`	Contraseña nueva, solo en el `.env` local
`DB_NAME`	Base de datos seleccionada en Aiven
`DB_SSL`	Mantiene TLS habilitado
`DB_CA_PATH`	Ruta opcional a la CA para validar el certificado

13. Conclusiones

Pay Services implementa un CRUD web sencillo conectado a una base MySQL real. La tabla observada, las seis pruebas unitarias, el ciclo CRUD, la persistencia tras reiniciar y la eliminación de datos temporales se verificaron. El servicio está desplegado en https://aiveneje3.onrender.com y `/api/health` confirmó que Aiven está conectado. El botón permite comprobar y reintentar la conexión después de la suspensión por inactividad del plan gratuito; no envía pings permanentes. Las capturas de la actividad aún deben agregarse a `evidencias/` y al PDF final.

14. Referencias técnicas

- Node.js, documentación oficial: https://nodejs.org/docs/
- Express 4.x, referencia oficial: https://expressjs.com/4x/api/
- MySQL2, guía de TLS: https://sidorares.github.io/node-mysql2/docs/documentation/ssl
- Aiven, certificados TLS/SSL: https://aiven.io/docs/platform/concepts/tls-ssl-certificates
- Aiven, conexión a MySQL: https://aiven.io/docs/products/mysql/howto/connect-from-mysql-workbench
- Render, límites del plan gratuito: https://render.com/docs/free
- Render, Web Services: https://render.com/docs/web-services
- Render, variables de entorno y secretos: https://render.com/docs/configure-environment-variables